NÍVEL DE TESTE · INTEGRAÇÃO

Teste de Integração

Teoria e prática: dos níveis de teste ao código, em Java e Python

Instrutor(a): **Hélcio Soares** · 21 de outubro de 2026

Continuação de: Testes Unitários

OBJETIVOS DA AULA

O que você vai aprender

1

Compreender os níveis de teste

Entender os níveis de teste de software — unitário, integração, sistema e aceitação — e como cada um se relaciona com o processo de desenvolvimento.

2

Aplicar testes de integração

Escrever, na prática, testes que verificam a comunicação real entre dois ou mais componentes do sistema — sem isolar tudo com dublês.

AGENDA

Como vamos usar a aula

≈ 45 min

Fundamentos

Níveis de teste de software e o que é um teste de integração

≈ 35 min

Mão na massa

Cenário de estoque: arquitetura, código e um teste de integração guiado

≈ 10 min

Trabalho prático

O desafio que vocês vão reproduzir sozinhos, com o roteiro de execução

≈ 5 min

Encerramento

Recapitulação e dúvidas

RELEMBRANDO A AULA ANTERIOR

A Pirâmide de Testes



Na aula passada, testamos a base da pirâmide: unidades isoladas.

Hoje subimos um nível: vamos testar como essas unidades **se comunicam de verdade** entre si e com a infraestrutura.

FUNDAMENTOS

Os níveis de teste de software

NÍVEL 1

Unitário

Testa uma unidade isolada (método/função). Rodado por quem desenvolve, o tempo todo.

NÍVEL 2 · HOJE

Integração

Testa a comunicação entre componentes (classes, serviços, banco). Rodado no pipeline de CI.

NÍVEL 3

Sistema (E2E)

Testa o sistema completo, do ponto de vista do usuário, antes de liberar para homologação.

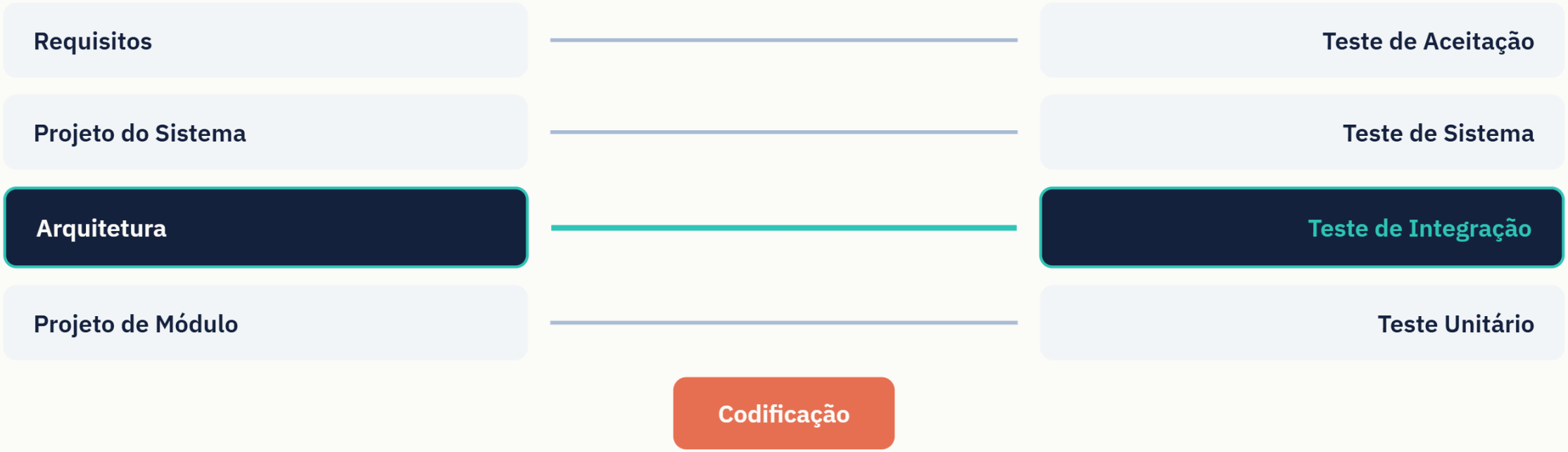
NÍVEL 4

Aceitação

Confirma que o sistema atende ao que o negócio pediu. Validado com o cliente ou PO.

ONDE CADA NÍVEL ENTRA NO CICLO

Definição de um lado, verificação do outro



DEFINIÇÃO

O que é um Teste de Integração?

*Um teste de integração verifica se dois ou mais componentes do sistema — classes, serviços, um banco de dados, uma API — funcionam **corretamente quando combinados**, e não apenas isoladamente.*

1

Comunicação real

Usa peças de verdade — ou o mais próximo disso — em vez de isolar tudo com dublês.

2

Pega o que o unitário não vê

SQL errado, mapeamento incompatível, configuração ausente, contrato quebrado entre módulos.

3

Mais lento, mais caro

Mexe com I/O real (banco, arquivo, rede) — por isso roda menos vezes que um teste unitário.

O PONTO CEGO DO TESTE UNITÁRIO

Por que só testes unitários não bastam?

Um teste unitário isola a classe com mocks e dublês. Isso prova que a **lógica** está certa — mas nada garante que as peças, juntas de verdade, continuam funcionando.

SQL errado

O mock do repositório sempre responde "certo". A query real, com erro de sintaxe ou nome de coluna, só falha contra o banco de verdade.

Mapeamento incompatível

Tipos de coluna, nulos ou formatos de data que não combinam entre o objeto Java/Python e a tabela real.

Configuração ausente

String de conexão, credencial ou variável de ambiente que só existe — ou falta — no ambiente real.

Contrato quebrado

Um módulo muda a assinatura ou o formato de resposta e o outro lado, testado isoladamente, nunca percebe.

ESTRATÉGIAS DE INTEGRAÇÃO

Big Bang vs. Incremental

Big Bang

Todos os módulos são integrados de uma vez só, no final do desenvolvimento.

Rápido de configurar, mas quando algo falha é difícil saber qual módulo causou o problema — tudo quebra ao mesmo tempo.

Incremental

Os módulos são integrados aos poucos, testando a cada novo passo.

Falhas ficam fáceis de isolar e o feedback é contínuo — é a abordagem recomendada na prática, inclusive em pipelines de CI.

INTEGRAÇÃO INCREMENTAL

Top-down, Bottom-up e Sandwich

Top-down

Integra de cima para baixo. Módulos inferiores ainda não prontos são substituídos por **stubs** (respostas falsas e simples).

Bottom-up

Integra de baixo para cima. Módulos superiores ainda não prontos são simulados por **drivers** (que chamam o módulo testado).

Sandwich

Combina os dois: integra a partir do meio, subindo e descendo ao mesmo tempo em direção às duas pontas.

GRANULARIDADE

Em que escopo um teste de integração atua?

Componente + Componente

Duas classes do seu próprio código que se chamam. Ex.: um Service que chama um Repository.

CENÁRIO DE HOJE

Componente + Infraestrutura

Código real conversando com um banco de dados, uma fila ou o sistema de arquivos.

Sistema + Sistema

Sua aplicação chamando uma API externa real ou outro serviço da empresa.

NO PIPELINE

Onde a integração entra no CI/CD?

1. Commit

Dev envia código.

2. Testes unitários

Rápidos, rodam a cada push, feedback em segundos.

3. Testes de integração

Mais lentos — sobem um banco real (ou H2/sqlite em memória) e validam a comunicação entre as peças.

4. Deploy

Só avança se as duas camadas de teste passarem.

Por isso a pirâmide: muitos testes unitários na base (rápidos e baratos), menos testes de integração no meio (mais lentos, mas pegam o que os unitários não veem).

BOAS PRÁTICAS

Quatro regras para testes de integração confiáveis

Ambiente isolado

Cada execução sobe seu próprio banco (ex.: H2 ou sqlite em memória). Nunca aponte testes para o banco de produção ou de outro dev.

Dados controlados

O teste cria os dados que precisa antes de rodar e sabe exatamente o que esperar — nada de depender de dados que "já estão lá".

Independência entre testes

Um teste não pode depender da ordem ou do resultado de outro. Cada teste prepara e limpa seu próprio estado.

Banco em memória quando possível

H2 (Java) ou sqlite :memory: (Python) dão velocidade de teste unitário com a realidade de um banco de verdade.

CAIXA DE FERRAMENTAS

O que vamos usar hoje

Java

JDK 17+ — a linguagem e a JVM

Maven — gerencia dependências e roda os testes

JUnit 5 — framework de testes

H2 Database — banco relacional em memória

JDBC — conecta o código ao banco

Python

Python 3.10+

pip — instala pacotes

pytest — framework de testes

sqlite3 — módulo padrão, banco em memória

Nenhuma instalação extra para o banco — já vem com o Python

PARTE PRÁTICA

Mão na massa

Vamos sair da teoria e escrever testes de integração de verdade, em Java e em Python, para um cenário de controle de estoque.

O CENÁRIO DE HOJE

Sistema de Controle de Estoque

Uma loja precisa saber, em tempo real, quanto tem de cada produto — e nunca vender mais do que existe. Vamos testar a integração entre a lógica de negócio e o banco de dados.

Produto

Entidade com id, nome e quantidade em estoque.

ProdutoRepository

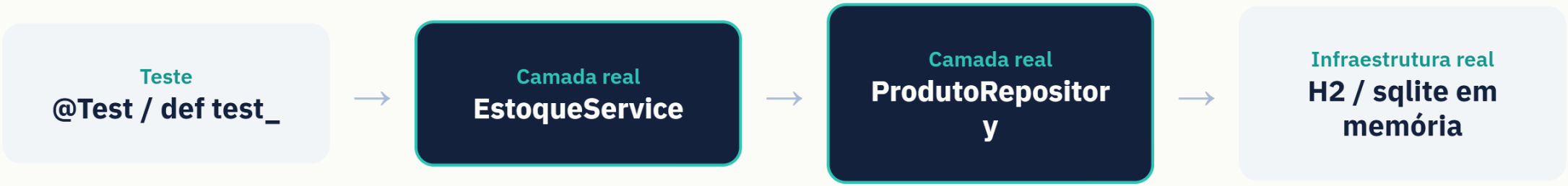
Lê e grava produtos no banco (JDBC em Java, sqlite3 em Python).

EstoqueService

adicionarEstoque e venderProduto — lança EstoqueInsuficienteException quando não há saldo.

ARQUITETURA

Onde o teste de integração entra



Nada é mockado a partir do Service para baixo — só o banco é trocado por uma versão em memória, para manter o teste rápido e isolado.

CÓDIGO FORNECIDO · 1 DE 2

ProdutoRepository — a ponte com o banco

ProdutoRepository.java

```
class ProdutoRepository {
   Connection conn;
   Produto buscarPorId(int id)
   void atualizarQuantidade(
      int id, int novaQtd)
   // usa JDBC + PreparedStatement
}
```

produto_repository.py

```
class ProdutoRepository:
   def __init__(self, conexao):
   def buscar_por_id(self, id):
   def atualizar_quantidade(
      self, id, nova_qtd):
   # usa sqlite3 + parâmetros ?
```

O código completo das duas classes está no roteiro do trabalho prático — aqui vemos só a forma.

CÓDIGO FORNECIDO · 2 DE 2

EstoqueService — a regra que vamos testar

EstoqueService.java

```java
void venderProduto(int id, int qtd) {
   Produto p = repo.buscarPorId(id);
   if (p.getQtd() < qtd)
      throw new EstoqueInsuficienteException();
   repo.atualizarQuantidade(id,
      p.getQtd() - qtd);
}
```

estoque_service.py

```python
def vender_produto(self, id, qtd):
   p = self.repo.buscar_por_id(id)
   if p.quantidade < qtd:
      raise EstoqueInsuficienteError()
   self.repo.atualizar_quantidade(
      id, p.quantidade - qtd)
```

Este método é o coração do exercício: ele só está correto se o **banco real** concordar com ele.

O BANCO DO TESTE

Esquema da tabela e setup em memória

schema.sql

```
CREATE TABLE produto (
   id INTEGER PRIMARY KEY,
   nome TEXT NOT NULL,
   quantidade INTEGER NOT NULL
);
```

Java — H2

```
jdbc:h2:mem:testdb
```

Python — sqlite3

```
sqlite3.connect(":memory:")
```

Cada execução de teste cria o banco do zero, roda o schema e descarta tudo ao final — sem arquivo, sem sujeira.

VAMOS SIMULAR JUNTOS

Escrevendo o primeiro teste de integração

Cenário: vender 3 unidades de um produto que tem 10 em estoque. O teste precisa provar que o banco realmente fica com 7.

1. Arrange

Cria o banco em memória, roda o schema e insere um produto com 10 unidades.

2. Act

Chama service.venderProduto(id, 3) — o método real, contra o banco real.

3. Assert

Lê o produto **de volta do banco** e confere que a quantidade é 7 — não confia em memória, confia no que foi persistido.

A diferença do teste unitário: aqui ninguém finge ser o banco. O banco é testado também.

RODANDO DE VERDADE

O que esperar no terminal

```
$ mvn test
Tests run: 3, Failures: 0,
Errors: 0, Skipped: 0
BUILD SUCCESS
```

```
$ pytest -v
test_adicionar_estoque... PASSED
test_vender_produto... PASSED
3 passed in 0.14s
```

Se um teste falhar aqui, o erro aponta para algo que só aparece com o banco real: SQL, tipo de coluna, ou uma transação que não foi commitada.

AGORA É COM VOCÊS

O trabalho prático

Vocês vão completar os testes que faltam, usando o mesmo cenário — sigam o roteiro de execução passo a passo.

O QUE FALTA FAZER

Três testes para vocês escreverem

1. **testAdicionarEstoqueAtualizaBanco** — adicionar unidades e confirmar no banco.

2. **testVenderProdutoComEstoqueSuficiente** — o caminho feliz, igual ao que fizemos juntos.

3. **testVenderProdutoComEstoqueInsuficienteNaoAlteraBanco** — o caminho de exceção: prova que, quando falha, o banco **não muda**.

COMO SABER QUE TERMINOU

Critérios de aceite do trabalho

- [ ] Os 3 testes existem e rodam sem erro de compilação ou import.
- [ ] Cada teste usa um banco em memória criado do zero (H2 ou sqlite).
- [ ] As asserções leem o estado **de volta do banco**, não de variáveis em memória.
- [ ] O teste de exceção confirma que o estoque **não foi alterado** quando a venda falha.
- [ ] mvn test ou pytest -v mostra todos os testes passando.

RECAPITULANDO

O que vocês levam desta aula

Objetivo 1

Os níveis de teste — unitário, integração, sistema, aceitação — e como cada um se encaixa no V-Model e no pipeline de CI/CD.

Objetivo 2

Como escrever um teste de integração de verdade, contra um banco real em memória, em Java e em Python.

Próximo passo: abram o roteiro de execução e completem os 3 testes pendentes. Bom trabalho!